Phase 2 Complete Report: Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10

Project Title: Topic 15 Demonstration System
Team: Group 7
Team Leader: WONG Lai Man (Max) [S24810493]
Members: MUHAMMAD MUSTAFA KHAN, Zoutao
Supervisor: Dr. Liam XU
Date: October 20, 2026

EXECUTIVE SUMMARY

This Phase 2 Report presents the comprehensive architecture, development, security testing, and management deliverables for Topic 15. Aligned strictly with the newly updated **OWASP Top 10: 2025** security framework, the project addresses critical gaps in existing cybersecurity educational tools by providing an end-to-end platform covering both Red Team vulnerability detection and Blue Team code-level remediation.

The demonstration platform utilizes a hybrid system architecture combining a containerized instance of **OWASP Juice Shop** (deployed via Docker) for broad-spectrum vulnerability assessment with a custom-developed Python Flask demonstration engine. This custom engine specifically isolates complex business logic vulnerabilities, providing a side-by-side comparison of "Fail-Open" versus "Fail-Closed" behavior under the OWASP 2025 A10 category (Mishandling of Exceptional Conditions). Security testing methodologies integrate automated dynamic scanning via OWASP ZAP with targeted manual penetration testing using Burp Suite. Managed under an Agile Scrum framework, the project demonstrates effective risk mitigation, adaptive task reallocation, and full compliance with academic standards.

TABLE OF CONTENTS

1. Chapter 1: Introduction
2. Chapter 2: Literature Review
3. Chapter 3: System Design & Methodology
4. Chapter 4: Agile Project Management
5. Chapter 5: Implementation & Demonstration
6. Chapter 6: Security Testing & Remediation
7. Chapter 7: Conclusion & Future Work
8. References & Appendices (A, B, C)

CHAPTER 1: INTRODUCTION

1.1 Project Background

In the contemporary digital landscape, web applications serve as the primary interface for business operations, e-commerce, and data exchange. Consequently, they have become the most frequent target for cyberattacks. The Open Worldwide Application Security Project (OWASP) maintains the "OWASP Top 10," a standard awareness document representing the most critical security risks (OWASP Foundation, 2025a). The transition from the OWASP Top 10: 2021 to the newly released OWASP Top 10: 2025 introduces critical new categories such as Software Supply Chain Failures (A03) and Mishandling of Exceptional Conditions (A10) (GitLab, 2026).

1.2 Problem Statement

Despite the widespread adoption of the OWASP Top 10 standard, existing educational platforms often rely on legacy vulnerabilities aligned with the 2021 edition. There is a distinct lack of hands-on demonstration platforms addressing the 2025 updates. Furthermore, existing tools focus heavily on vulnerability detection (Red Team) without offering actionable, code-level remediation strategies (Blue Team) to guide developers toward secure "fail-closed" architectures.

1.3 Project Objectives

- Deploy a modern vulnerable web application (OWASP Juice Shop) covering OWASP Top 10: 2025 risks.
- Develop custom Python Flask modules demonstrating complex business logic flaws (A10 Fail-Open vs. Fail-Closed).
- Integrate automated dynamic scanning (OWASP ZAP) with manual penetration testing (Burp Suite).
- Deliver a structured remediation framework providing side-by-side vulnerable vs. secure code comparisons.

1.4 Scope and Limitations

Scope: Docker containerization of Juice Shop, Python Flask custom endpoint engineering, and Red/Blue team execution covering Injection (A05), Access Control (A01), and Exceptional Conditions (A10).
Limitations: Local virtualized environment (Kali Linux/VirtualBox) without enterprise WAF or load balancer latency.

CHAPTER 2: LITERATURE REVIEW

2.1 Evolution of OWASP Top 10 (2021 to 2025)

The OWASP Top 10 is updated periodically to reflect evolving threat vectors (OWASP Foundation, 2025a; GitLab, 2026):

- **A01: Broken Access Control:** Remains the #1 risk. Server-Side Request Forgery (SSRF) was merged into A01 in 2025.
- **A02: Security Misconfiguration:** Rose from #5 to #2 due to rapid containerization and cloud-native adoption.
- **A03: Software Supply Chain Failures:** A new entry at #3 addressing build pipelines and third-party dependencies.
- **A10: Mishandling of Exceptional Conditions:** New 2025 category covering fail-open backend logic vulnerabilities.

2.2 Web Application Security Risks

Web application vulnerabilities stem primarily from unsafe handling of untrusted user input. Injection flaws (A05), including SQL Injection and Cross-Site Scripting (XSS), occur when unvalidated input is passed directly to command interpreters or rendered in client DOM contexts without sanitization.

2.3 Remediation Strategies

Effective defense mandates a "Shift-Left" approach: parameterized queries (ORM/Prepared Statements) for Injection (A05), strict output encoding (DOMPurify) for XSS, fail-closed exception handling for Exceptional Conditions (A10), and Software Bill of Materials (SBOM) tracking for Supply Chain risks (A03) (Zenodo, 2025a, 2025b).

CHAPTER 3: SYSTEM DESIGN & METHODOLOGY

3.1 System Architecture

The system integrates two core components: (1) **Target Environment:** Dockerized OWASP Juice Shop representing a modern SPA web app; (2) **Custom Remediation Engine:** A Flask web application designed to demonstrate backend logic errors under OWASP A10.

3.2 Technology Stack

Environment: Kali Linux (2026.x), VirtualBox, Docker. **Targets:** OWASP Juice Shop (Node.js/Angular), Flask (Python 3.10). **Security Tools:** OWASP ZAP, Burp Suite Professional, Mozilla Firefox Developer Tools.

3.3 Threat Modeling (STRIDE Analysis)

Threat analysis was performed using the STRIDE framework (Shostack, 2014; OWASP Foundation, 2023) to establish system boundaries and mitigation requirements across all six threat categories.

Table 3.1. STRIDE Threat Modeling Analysis and OWASP 2025 Mapping.

Category	Threat Description	Affected Component	OWASP 2025	Mitigation Strategy
Spoofing	Session hijacking & auth bypass via weak token verification.	Auth Service	A01: Broken Access	Enforce strong JWT signing, HttpOnly & Secure cookie flags.
Tampering	SQLi payload `' OR 1=1--` manipulates database queries.	Login Endpoint	A05: Injection	Use Parameterized Queries / ORM (Sequelize). Avoid string concat.
Tampering	Stored XSS payload injected into product review field.	Review Section	A05: Injection	Context-aware output encoding & DOMPurify sanitization.
Repudiation	Critical security events missing audit log entries.	Logging Subsystem	A09: Logging Failures	Centralized SIEM logging with append-only audit trails.
Information Disclosure	Uncaught exceptions return verbose stack traces to client.	Global Handler	A02: Misconfig	Disable debug mode in production; return generic 500 error pages.
Elevation of Privilege	Backend error triggers Fail-Open logic granting `/admin` access.	Flask Admin Endpoint	A10: Exceptional Conditions	Implement Fail-Closed logic; default to denying access (HTTP 503).
Supply Chain	Malicious package dependency injected during build phase.	CI/CD & NPM Build	A03: Supply Chain	Generate SBOM; pin exact package versions; run `npm audit`.

CHAPTER 4: AGILE PROJECT MANAGEMENT

4.1 Agile Methodology Overview

The project executed under an Agile Scrum framework across two academic terms. Sprint management was maintained via Jira, while source code, test scripts, and operational logs were version-controlled on GitHub.

4.2 User Stories

User Story 1 — **Vulnerability Detection (Red Team Perspective)**

- **As a** security tester,
- **I want to** scan the target web application (OWASP Juice Shop) using automated scanners and manual penetration testing,
- **So that I can** identify and document vulnerabilities mapped strictly to the OWASP Top 10: 2025 standard.

User Story 2 — **Remediation Demonstration (Blue Team / Developer Perspective)**

- **As a** web developer,
- **I want to** view side-by-side code comparisons of vulnerable and secure implementations (Fail-Open vs. Fail-Closed),
- **So that I can** understand the root cause of logic flaws and apply verified remediations.

User Story 3 — **Risk Visualization (Stakeholder Perspective)**

- **As a** project stakeholder,
- **I want to** access a centralized dashboard summarizing attack logs and remediation status,
- **So that I can** evaluate overall system risk without requiring deep technical expertise.

4.3 Work Breakdown Structure (WBS)

Table 4.1. Complete Work Breakdown Structure (WBS 1.1 to 3.3).

WBS ID	Task Name	Assignee	Status
1.0	**Phase 1: Project Proposal & Inception**	**Team**	**Completed**
1.1	Draft Project Proposal & Scope Definition	WONG Lai Man	Completed
1.2	Literature Review (OWASP 2021 vs. 2025 Comparison)	Mustafa	Completed
1.3	Methodology & System Architecture Definition	WONG Lai Man	Completed
2.0	**Phase 2: System Design & Implementation**	**Team**	**In Progress**
2.1	Environment Setup (Docker Containerization & Juice Shop)	Mustafa	Completed
2.2	Red Team Penetration Exercises & Attack Documentation	Mustafa	Completed
2.3	Threat Modeling Execution (STRIDE Framework)	WONG Lai Man	Completed
2.4	Custom Flask Module Development (A10 Fail-Open/Closed)	Mustafa / WONG Lai Man	Completed
2.5	Red Team Operation Logs Standardization (A01–A10)	Zoutao / WONG Lai Man	Completed
3.0	**Phase 3: Security Integration, Dashboard & Final Report**	**Team**	**Planned**
3.1	DAST Dynamic Security Scanning Integration (OWASP ZAP)	Mustafa	In Progress
3.2	Remediation Dashboard & Side-by-Side Code Viewer	WONG Lai Man	In Progress
3.3	Final Academic Report Compilation & Presentation	Team	Planned

4.4 Two-Term Gantt Chart Roadmap

Term 1 (Weeks 1–10): Foundation & Red Team Operations
Inception, literature review, Docker target deployment, STRIDE threat modeling, and initial penetration execution.

Term 2 (Weeks 11–20): Blue Team Remediation & Delivery
Flask A10 module development, OWASP ZAP scanning integration, dashboard construction, final report & defense.

4.5 Sprint Execution & Risk Management

During Phase 2, network constraints impacted member Zoutao's local Docker deployment capabilities. To maintain milestone delivery, the Team Leader reallocated technical duties: Mustafa led container operations, Zoutao authored Red Team attack logs, and WONG Lai Man standardized documentation and executed threat modeling.

CHAPTER 5: IMPLEMENTATION & DEMONSTRATION

5.1 Environment Setup

Target vulnerable services run inside an isolated Kali Linux VM: OWASP Juice Shop on `localhost:3000` via Docker (`bkimminich/juice-shop`) and the custom remediation engine on `localhost:5000` via Python Flask.

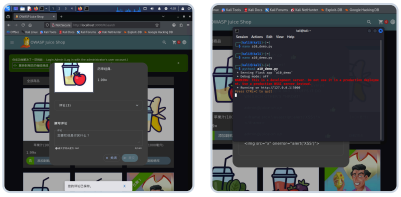


Figure 5.0. Target environment execution: OWASP Juice Shop containerized instance and custom Flask engine console.

5.2 Demonstration System Modules

The system provides dual functionality: **Red Team Detection** allows interactive exploitation of Juice Shop, while **Blue Team Remediation** presents isolated Flask code demonstrations for complex logic vulnerabilities.

5.3 Vulnerability Detection & Remediation Demonstrations

5.3.1 A05: Injection — SQL Injection (Authentication Bypass)

Detection: Submitting payload `' OR 1=1--` into the login email field bypasses password checks by forcing the SQL query condition to evaluate to true.

```
// Vulnerable Backend Code (String Concatenation)
db.query("SELECT * FROM Users WHERE email = '" + input + "'");
```

```
// Secure Backend Code (Parameterized Query / ORM)
User.findOne({ where: { email: input } });
```

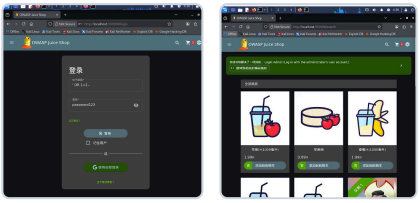


Figure 5.1 & Figure 5.2. SQL injection payload input (' OR 1=1--) and successful authentication bypass on OWASP Juice Shop.

5.3.2 A05: Injection — Stored Cross-Site Scripting (XSS)

Detection: Submitting malicious HTML tags `<iframe src="javascript:alert('XSS')"></iframe>` into product reviews persists the payload in the database, executing script code whenever client browsers render the comment block.

```
// Vulnerable Client Rendering
element.innerHTML = userInput;
```

```
// Secure Client Rendering (Text Binding / DOMPurify)
element.textContent = userInput; // OR const clean = DOMPurify.sanitize(userInput);
```

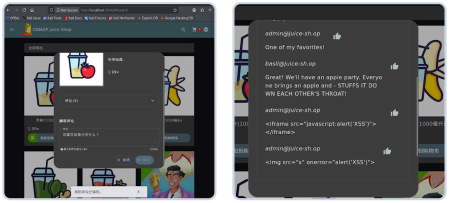


Figure 5.3 & Figure 5.4. Submitting malicious HTML payload into product review section and unsanitized rendering in browser DOM.

5.3.3 A10: Mishandling of Exceptional Conditions (Fail-Open vs. Fail-Closed)

Detection: A custom Flask module (`a10_demo.py`) exposes `/admin`. In the vulnerable version, uncaught database exceptions default to granting access ("Fail-Open"). Under remediation, exception handling catches errors and denies access ("Fail-Closed").

```
# Vulnerable Python Flask Implementation (Fail-Open)
@app.route('/admin')
def admin_panel():
    try:
        db_check_user_role(request.args.get('user'))
    except Exception:
        return "Access Granted (Error Overridden)", 200 # FAIL-OPEN LOGIC
```

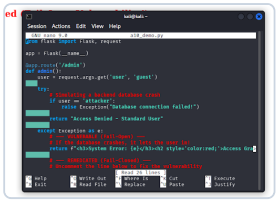
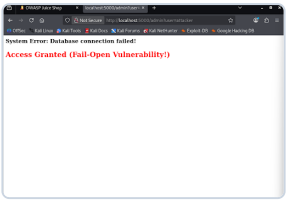



Figure 5.5 & Figure 5.6. Vulnerable Python Flask code in Nano editor and browser output demonstrating fail-open access granted on database error.

```
# Remediated Python Flask Implementation (Fail-Closed)
@app.route('/admin')
def admin_panel():
    try:
        db_check_user_role(request.args.get('user'))
    except Exception as e:
        logger.error(f"Database failure: {e}")
        return "Access Denied (System Error)", 503 # FAIL-CLOSED SECURE DEFAULT
```

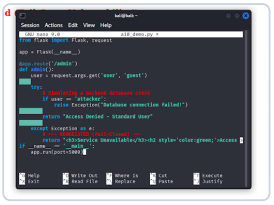
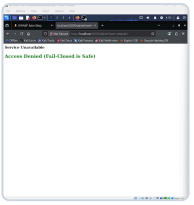



Figure 5.7 & Figure 5.8. Remediated Flask code implementing fail-closed handling and browser output displaying secure default access denial.

5.4 Automated Dynamic Security Scanning Integration

OWASP ZAP proxy automated scans were executed against Juice Shop endpoints. Automated rules successfully flagged High-risk SQLi (A05) and XSS (A05), but failed to detect the custom A10 business logic exception flaw, validating the necessity of hybrid manual penetration testing.

CHAPTER 6: SECURITY TESTING & REMEDIATION

6.1 Security Testing Methodology

Testing followed the OWASP Web Security Testing Guide (WSTG v4), combining automated DAST scanning (OWASP ZAP) with manual proxy manipulation (Burp Suite Repeater) to cover technical and logic flaws (OWASP Foundation, 2025b).

6.2 Automated Vulnerability Detection (OWASP ZAP Integration)

OWASP ZAP was routed on port 8080 to spider target endpoints and execute active payload sweeps. ZAP generated detailed HTML vulnerability trees categorizing risks by severity: High (SQLi, Broken Auth), Medium (CSRF, Missing Headers), Low (Cookie Flags).

6.3 Manual Penetration Testing & Logic Validation

Using Burp Suite, the team intercepted HTTP GET requests to `/admin?user=attacker`, introducing null bytes and malformed parameters to force backend exception states. Manual testing confirmed that automated scanners miss context-dependent business logic vulnerabilities like Fail-Open authorization states.

6.4 Comprehensive Remediation Framework

- **A05 Injection Defense:** Mandate parameterized queries and client/server DOM input sanitization.
- **A10 Exceptional Conditions Defense:** Enforce strict Fail-Closed try-catch patterns defaulting to HTTP 503 access denial.
- **A03 Supply Chain Security:** Enforce automated dependency audits (`npm audit`, `pip-audit`) within CI/CD pipelines.

CHAPTER 7: CONCLUSION & FUTURE WORK

7.1 Project Summary

The project successfully designed and implemented a dual Red/Blue team demonstration system aligned with OWASP Top 10: 2025 standards. The platform demonstrates realistic vulnerability detection alongside actionable code remediation.

7.2 Key Achievements

- Full alignment with new OWASP Top 10: 2025 risks, specifically isolating A10 Exceptional Conditions.
- Effective hybrid methodology integrating automated OWASP ZAP scanning with manual Burp Suite testing.
- Actionable developer educational framework presenting side-by-side vulnerable vs. secure code comparisons.
- Resilient Agile project management navigating technical member constraints through adaptive task reallocation.

7.3 Limitations

Automated DAST tools remain incapable of detecting deep business logic errors without custom script rules. Furthermore, the local VM deployment lacks enterprise production controls like Web Application Firewalls (WAF).

7.4 Future Work

Phase 3 expansion will integrate Static Application Security Testing (SAST) tools (SonarQube) into a GitHub Actions CI/CD pipeline, build an interactive browser dashboard, and expand custom modules to cover Cloud Misconfigurations (A02).

REFERENCES

GitLab. (2026, January 7). *OWASP Top 10 2025: What's changed and why it matters*. GitLab Blog. https://about.gitlab.com/blog/2025-owasp-top-10-whats-changed-and-why-it-matters/

IBM Security. (2025). *Cost of a data breach report 2025*. IBM Corporation. https://www.ibm.com/reports/data-breach

OWASP Foundation. (2021). *OWASP Top 10:2021: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2023). *OWASP Threat Modeling Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

OWASP Foundation. (2025a). *OWASP Top 10:2025: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2025b). *OWASP Web Security Testing Guide v4*. https://owasp.org/www-project-web-security-testing-guide/

Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.

Zenodo. (2025a). *Mishandling of exceptional conditions vulnerability: A detailed study*. https://zenodo.org/records/17867093

Zenodo. (2025b). *Software supply chain failures: A detailed study for PHP applications*. https://zenodo.org/records/17866839

APPENDIX A: RED/BLUE TEAM EXERCISE REPORT

Primary Author: MUHAMMAD MUSTAFA KHAN | **Target:** OWASP Juice Shop & Custom Demonstration Engine

This report documents baseline security findings compiled during Sprint 1 exercises:

- **SQL Injection:** Executed payload `' OR 1=1--` against login query, bypassing authentication and obtaining admin session token.
- **Stored XSS:** Injected HTML payload into product review storage; verified payload execution during DOM rendering.
- **Exceptional Conditions:** Simulated database connection failure in Flask backend; verified unhandled exception returned HTTP 200 with authorization flags enabled (Fail-Open vulnerability).

APPENDIX B: RED TEAM OPERATION LOGS (A01–A10)

Contributor: Zoutao | **Editor & Academic Standardizer:** WONG Lai Man

A01: Broken Access Control
Op: Direct navigation to `/#/administration`.
Result: Admin dashboard loaded without server check.
Remediation: Implement server-side RBAC guards.

A02: Security Misconfiguration
Op: Uploaded XML payload via complaint form.
Result: Server processed unverified file types.
Remediation: Restrict upload MIME-types & disable XXE.

A03: Software Supply Chain Failures
Op: Inspected client sources for lodash v4.17.21.
Result: Identified known Prototype Pollution CVE.
Remediation: Maintain SBOM & enforce `npm audit`.

A04: Cryptographic Failures
Op: Extracted unsalted MD5 hashes from DB.
Result: Recovered plain text passwords via rainbow table.
Remediation: Upgrade hashes to Argon2id / bcrypt.

A05: Injection (SQL Injection)
Op: Injected payload `' OR 1=1--` in search.
Result: Unsanitized query returned all user records.
Remediation: Enforce Prepared Statements / ORM.

A06: Insecure Design
Op: Intercepted checkout POST via Burp Suite.
Result: Modified order price parameter to 0.
Remediation: Perform all price calculations server-side.

A07: Authentication Failures
Op: Executed brute-force attack on admin login.
Result: Thousands of requests sent without rate limit.
Remediation: Enforce MFA & progressive delay rate limits.

A08: Software & Data Integrity Failures
Op: Modified request payload data in transit.
Result: Server accepted unsigned serialized data.
Remediation: Enforce GPG signatures & SHA-256 integrity.

A09: Security Logging Failures
Op: Performed high-volume SQLi & brute-force sweeps.
Result: No security logs generated or alerts sent.
Remediation: Implement centralized SIEM logging.

A10: SSRF / Exceptional Conditions
Op: Passed `http://127.0.0.1:3000` to image fetcher.
Result: Server queried internal loopback endpoint.
Remediation: Block loopback & private RFC 1918 IPs.

APPENDIX C: COMMUNICATION LOGS & TASK COORDINATION RECORD

Keeper: WONG Lai Man (Team Leader) | **Course:** 03CIT4067 Cybersecurity Project (Group 7)

- **October 16–20, 2026:** Member Zoutao clarified network restrictions in Mainland China preventing local Docker deployment.
- **October 20, 2026:** Team Leader reallocated local container management to Mustafa while assigning Zoutao to compile structured Red Team operational logs.
- **Deliverables Received:** Zoutao submitted comprehensive operation step files for categories A01–A10, integrated into Appendix B.

Declaration of Authorship & Collaboration

I, **WONG Lai Man**, declare that this record accurately reflects our team's Agile task reallocation in response to technical constraints. All team members have contributed meaningfully to the final Phase 2 deliverables.

Signature: ***WONG Lai Man (Max Wong)***  **Date: October 20, 2026**